Part 3 – Scenario 10: Rover Firmware Queue

Scenario Explanation

The rover needs a priority queue that supports insertion, finding and removing the minimum key, melding queues, and decreasing a key. Each node is restricted to three fields: its key, first-child pointer, and next-sibling pointer. Extract-min must recombine the removed root's subtrees efficiently and count the links used.

Choice and Justification

A pairing heap fits the two-pointer node layout and uses a multiway heap structure. Insert and meld link roots; find-min reads the root. Extract-min uses two-pass pairing to recombine the removed root's children. A conventional Fibonacci heap generally needs extra pointers and metadata, conflicting with the strict node-size limit. A binary heap is compact but does not naturally support O(1) meld.

Complexity Analysis

Operation	Implementation / cost
Insert	Create a node and link it to the root; O(1) actual time.
Meld	Link the two roots and empty the second heap; O(1) actual time.
Find-min	Read the root key; O(1) actual time.
Extract-min	Pair adjacent child heaps, then combine the paired heaps in a second pass; count every consolidation link.
Decrease-key	Lower the key, search for its parent, detach it, and link it to the root. Parent search is O(n) worst case.
Space	O(n) heap nodes plus reference-queue storage.

Observation

The pairing heap follows the required node layout and supports the requested operations. Two-pass consolidation recombines the removed root's subtrees and counts its links. The reference priority queue checks correctness independently.

The key trade-off is decrease-key: omitting parent pointers saves node space but makes finding the parent require traversal. Include the actual link count and a zero-failure validation result from output.txt in the final submission.